

Lab 04 - Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador

Lab id	Lab 04
Day	1 - Cloud Native Foundations & Kubernetes Core Concepts
Duration	50 minutes
Namespace	kcna-lab04
Mapping	LO1 Develop a Kubernetes architectural proof of concept A1 Develop an architectural proof of concept K1 Process for developing proof of concepts

1. Objective

By the end of this lab you will be able to:

- State exactly what containers in a Pod share - one network namespace and named volumes - and what they do not share.
- Build and explain the three classic composite-container patterns: sidecar, adapter and ambassador.
- Use `kubect! logs -c <container>` and `kubect! exec -c <container>`, and recognise the error you get when you forget `-c`.
- Use a native sidecar - an `initContainer` with `restartPolicy: Always`, stable since Kubernetes v1.29 - and explain the ordering guarantee it buys.
- Diagnose a Pod rejected at admission because two containers did not really share the volume the author thought they shared.

2. Prerequisites

- Kubernetes v1.29 or later (native sidecars); this lab is written against v1.30+. `kubect!` on PATH.
- Labs 01-03 completed.

```
kubect! version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kcna-qa-20260905-control-plane	Ready	control-plane	88m	v1.37.0

```
cd courseware/labs/lab-04-multi-container-patterns
```

3. Scenario

The Depot Portal booking API at Meridian Freight Pte Ltd has three requirements that the application team refuses to build into the application itself, and they are right to refuse:

1. Logs must reach the central platform. Today they are written to a file on disk in Meridian's own key=value format.
2. Metrics must be scrapeable by the platform's Prometheus. The application emits no metrics at all and will not be rewritten this quarter.
3. Outbound calls to the tariff service must gain retries and, later, mutual TLS. The application hard-codes `http://127.0.0.1:9000` and cannot be changed.

Every one of those is solvable outside the application, in the same Pod. That is what composite-container patterns are for, and it is one of the strongest arguments in the PoC for adopting Kubernetes.

4. Step-by-step procedure

Step 1 - What a Pod actually is

A Pod is not "a container". It is a shared execution context: one or more containers that share

- one network namespace - one Pod IP, one loopback interface, one port space;
- one set of named volumes, each mounted where and how each container asks;
- one lifecycle, one node, one scheduling decision.

They do not share a filesystem root, and by default they do not share a PID namespace. That is the entire mental model, and everything below follows from it.

```
kubectl explain pod.spec.containers.volumeMounts.name
```

```
GROUP:
KIND:      Pod
VERSION:   v1

FIELD: name <string> -required-

DESCRIPTION:
  This must match the Name of a Volume.
```

"This must match the Name of a Volume" - remember that sentence; it becomes the failure injection in section 6.

Step 2 - Namespace, datasets and the remote service

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab04 created
```

Look at the seed dataset - 16 rows of Meridian's proprietary access log:

```
head -n 3 data/access-seed.log
```

```
2026-09-05T08:00:01Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=41
2026-09-05T08:00:04Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=38
2026-09-05T08:00:07Z depot=MF-SIN-02 path=/api/bays status=200 bytes=1544 latency_ms=52
```

```
kubectl -n kcna-lab04 create configmap access-seed --from-file=data/access-seed.log
kubectl -n kcna-lab04 create configmap tariff-quote --from-file=data/tariff-quote.json
```

```
configmap/access-seed created
configmap/tariff-quote created
```

Bring up the remote tariff service that the ambassador will proxy to:

```
kubectl apply -f manifests/10-tariff-backend.yaml
```

```
pod/tariff-backend created
service/tariff-backend created
```

```
kubectl -n kcna-lab04 wait --for=condition=Ready pod/tariff-backend --timeout=120s
kubectl -n kcna-lab04 get endpointslices -l kubernetes.io/service-name=tariff-backend
```

```
pod/tariff-backend condition met
NAME                                ADDRESSSTYPE  PORTS  ENDPOINTS  AGE
tariff-backend-9k4xd              IPv4          80     10.244.0.19  22s
```

Use endpointslices, not endpoints. The v1 Endpoints resource was

deprecated in Kubernetes v1.33 in favour of discovery.k8s.io/v1

EndpointSlice.

Step 3 - Pattern 1: SIDECAR (a log shipper on a shared volume)

Definition. A sidecar *adds a capability* the application does not have, without changing the application.

```
kubectl apply -f manifests/20-sidecar-logship.yaml
```

```
pod/depot-sidecar created
```

```
kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-sidecar --timeout=120s
kubectl -n kcna-lab04 get pod depot-sidecar
```

```
pod/depot-sidecar condition met
NAME          READY   STATUS    RESTARTS   AGE
depot-sidecar 2/2     Running   0           25s
```

READY 2/2 - the first number is ready containers, the second is total. Now ask for the logs the way you did in Lab 02:

```
kubectl -n kcna-lab04 logs depot-sidecar
```

```
error: a container name must be specified for pod depot-sidecar, choose one of: [app shipper]
```

kubectl logs streams one container. With more than one it cannot guess:

```
kubectl -n kcna-lab04 logs depot-sidecar -c app --tail=3
```

```
[app] seeded 16 historical rows
```

```
kubectl -n kcna-lab04 logs depot-sidecar -c shipper --tail=6
```

```
2026-09-05T08:00:37Z depot=MF-SIN-02 path=/api/trailers status=404 bytes=64 latency_ms=11
2026-09-05T08:00:40Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=37
2026-09-05T08:00:43Z depot=MF-SIN-02 path=/api/tariff status=503 bytes=0 latency_ms=3002
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:12:07Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
2026-09-05T16:12:10Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

Stop and read that. The shipper container never wrote a single one of those lines. It is emitting content produced by a *different container* - the first four rows came from data/access-seed.log, the last two were generated live - because both containers mount the same emptyDir volume named varlog.

That log stream is now an ordinary container log, so anything that collects container logs (a DaemonSet log agent, the cloud provider's collector) picks it up for free. That is the sidecar's whole value.

Prove the two containers have separate filesystems but one shared directory:

```
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls -l /var/log/depot
```

```
total 4
-rw-r--r--  1 1000    3000          2814 Sep  5 16:12 access.log
```

```
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls /seed
```

```
ls: /seed: No such file or directory
command terminated with exit code 1
```

/seed exists only in the app container - it declared that mount, the shipper did not. Same Pod, different filesystems.

Now try to write from the shipper:

```
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- \
sh -c 'echo tampered >> /var/log/depot/access.log'
```

```
sh: can't create /var/log/depot/access.log: Read-only file system
command terminated with exit code 1
```

The same volume is mounted readOnly: true in the shipper and read-write in the app. Mount options are per container, not per volume.

Finally, look at how the emptyDir is bounded:

```
kubectl -n kcna-lab04 get pod depot-sidecar \
-o jsonpath='{.spec.volumes[?(@.name=="varlog")].emptyDir.sizeLimit}{"\n"}'
```

```
16Mi
```

An unbounded emptyDir can fill the node's disk and get your Pod evicted. Always set sizeLimit.

Step 4 - Pattern 2: ADAPTER (reshape an output to a standard)

Definition. An adapter **changes the shape** of something the application already produces, so that a standard consumer can read it. The application is unchanged and unaware.

```
kubectl apply -f manifests/30-adapter-metrics.yaml
```

```
pod/depot-adapter created
```

```
kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-adapter --timeout=120s
```

```
pod/depot-adapter condition met
```

The app still speaks Meridian's private format:

```
kubectl -n kcna-lab04 logs depot-adapter -c app --tail=2
```

```
2026-09-05T16:14:22Z depot=MF-SIN-02 path=/api/bookings status=404 bytes=812 latency_ms=44
2026-09-05T16:14:24Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

Wait about 20 seconds for the adapter's loop to fire, then:

```
kubectl -n kcna-lab04 logs depot-adapter -c adapter --tail=12
```

```
# HELP depot_http_requests_total Depot Portal requests by status code.
# TYPE depot_http_requests_total counter
depot_http_requests_total{depot="MF-SIN-02",code="200"} 24
depot_http_requests_total{depot="MF-SIN-02",code="201"} 4
depot_http_requests_total{depot="MF-SIN-02",code="404"} 4
depot_http_requests_total{depot="MF-SIN-02",code="503"} 4
# HELP depot_http_requests_observed_total All parsed log rows.
# TYPE depot_http_requests_observed_total counter
depot_http_requests_observed_total{depot="MF-SIN-02"} 36
```

That is the Prometheus text exposition format - # HELP, # TYPE, then metric{label="value"} number. You will scrape exactly this shape in Lab 24. The application produced none of it.

Sidecar or adapter? The mechanics are identical - an extra container on a shared volume. The *intent* differs, and that is what you write in the design document:

	Sidecar	Adapter	Ambassador
Direction	augments the app	reshapes app output	owns app outbound traffic
App awareness	none	none	none (it just calls localhost)
Typical use	log shipping, config reload, cert rotation	metrics exporter, log reformatter, protocol translation	proxy, TLS origination, retries, sharding
Real-world example	Fluent Bit shipping a log file	*_exporter translating to Prometheus	Envoy in a service mesh

Step 5 - Pattern 3: AMBASSADOR (own the outbound connection)

Definition. An ambassador *represents the outside world to the application on localhost*. The application makes the simplest possible call; the ambassador deals with discovery, TLS, retries and topology.

```
kubectl apply -f manifests/40-ambassador-proxy.yaml
```

```
configmap/ambassador-conf created
pod/depot-ambassador created
```

```
kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-ambassador --timeout=120s
kubectl -n kcna-lab04 get pod depot-ambassador
```

```
pod/depot-ambassador condition met
NAME                READY   STATUS    RESTARTS   AGE
depot-ambassador    2/2     Running   0           41s
```

```
kubectl -n kcna-lab04 logs depot-ambassador -c app --tail=4
```

```
[app] I only know about http://127.0.0.1:9000
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
[app] quote via ambassador: "bay_hour":51.00,
```

The bay_hour value came from data/tariff-quote.json, which lives in a different Pod (tariff-backend), reached through a Service DNS name the application has never heard of. Look at the proxy's own log:

```
kubectl -n kcna-lab04 logs depot-ambassador -c ambassador --tail=3
```

```
127.0.0.1 - - [05/Sep/2026:16:16:02 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
127.0.0.1 - - [05/Sep/2026:16:16:12 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
127.0.0.1 - - [05/Sep/2026:16:16:22 +0000] "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "wget"
```

The client address is 127.0.0.1. Two containers, one loopback interface - because they share a network namespace. Confirm that they also share one IP:

```
kubectl -n kcna-lab04 exec depot-ambassador -c app -- \
  wget -qO- http://127.0.0.1:9000/ambassador-health
```

```
ambassador ok
```

Because the port space is shared, two containers in one Pod may not both bind the same port. Read the proxy configuration that made this work:

```
kubectl -n kcna-lab04 get configmap ambassador-conf \
-o go-template='{{index .data "default.conf"}}' | head -n 12
```

```
server {
    listen      9000;
    server_name localhost;

    # Everything the application asks for on localhost:9000 is forwarded
    # to the real, cluster-internal service.
    location / {
        proxy_pass      http://tariff-backend.kcna-lab04.svc.cluster.local:80;
        proxy_set_header Host          $host;
        proxy_set_header X-Meridian-Via ambassador;
        proxy_connect_timeout 2s;
        proxy_read_timeout   5s;
    }
}
```

Adding mutual TLS, retries or a canary split later means editing that ConfigMap - not the application. This is precisely how a service mesh sidecar (Envoy in Istio or Linkerd) works, injected automatically instead of written by hand.

Step 6 - Native sidecars (Kubernetes v1.29+)

The three patterns above all use ordinary spec.containers, and they share one weakness: start-up order is not guaranteed. If the shipper starts after the app, early log lines can be lost; if an ambassador starts after the app, the app's first outbound calls fail.

Kubernetes v1.29 made restartable init containers - "native sidecars" - available by default, and the feature reached GA in v1.33. An initContainer with restartPolicy: Always:

- starts before any regular container, in list order;
- keeps running for the life of the Pod (it does not have to exit);
- is not counted in the Pod's READY n/m ratio;
- does not stop a Job's Pod from reaching Completed;
- is terminated after the regular containers, so it can flush buffers.

```
kubectl apply -f manifests/50-native-sidecar.yaml
```

```
pod/depot-native-sidecar created
```

```
kubectl -n kcna-lab04 wait --for=condition=Ready pod/depot-native-sidecar --timeout=120s
kubectl -n kcna-lab04 get pod depot-native-sidecar
```

```
pod/depot-native-sidecar condition met
NAME                READY   STATUS    RESTARTS   AGE
depot-native-sidecar 1/1     Running   0           33s
```

1/1, not 2/2 - the sidecar is not in spec.containers. It is running all the same:

```
kubectl -n kcna-lab04 get pod depot-native-sidecar \
-o jsonpath='{range .status.initContainerStatuses[*]}{.name}{"\t"}{.state}{"\n"}{end}'
```

```
prepare{"terminated":{"exitCode":0,"finishedAt":"2026-09-05T16:17:41Z","reason":"Completed",
"startedAt":"2026-09-05T16:17:41Z"}}
shipper{"running":{"startedAt":"2026-09-05T16:17:43Z"}}
```

Two init containers, two completely different outcomes:

- prepare has no restartPolicy, so it is a classic init container: it ran

to completion (exitCode: 0) and the Pod moved on.

- shipper has restartPolicy: Always, so it is running and will keep

running.

```
kubectl -n kcna-lab04 logs depot-native-sidecar -c shipper --tail=3
```

```
[shipper] native sidecar up BEFORE the app container
2026-09-05T08:00:46Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=42
2026-09-05T16:17:48Z depot=MF-SIN-02 path=/api/bookings status=200 bytes=812 latency_ms=44
```

-c shipper works for init containers too - no special flag needed.

	Classic init container	Native sidecar	Regular container
Field	initContainers[]	initContainers[] + restartPolicy: Always	containers[]
Must exit	yes, 0	no	no
Start order	before everything	before regular containers, in list order	no guarantee among peers
Counted in READY n/m	no	no	yes
Blocks Job completion	no	no	yes
Since	v1.0	v1.29 (GA v1.33)	v1.0

Choose native sidecars for anything the application depends on at start-up - log shippers, proxies, credential agents - and especially inside Jobs, where an old style sidecar would keep the Job running forever.

5. Verification

```
chmod +x verification/checks.sh
bash verification/checks.sh
```

The script checks the namespace and both ConfigMaps; that tariff-backend is Ready and has an EndpointSlice; that depot-sidecar has two containers and that the shipper re-emitted the 16 seeded rows it never wrote; that the adapter is producing Prometheus # TYPE metadata and a 503 series; that the ambassador's app got a quote through 127.0.0.1:9000 and the proxy logged a 200; that the native sidecar declares restartPolicy: Always and is in a *running* state while the app runs; and finally that the broken manifest is rejected at admission (via --dry-run=server, so nothing is created).

Expected tail:

```
Result: 20 passed, 0 failed
```

Full annotated evidence: [`verification/expected-output.md`](#).

6. Failure injection - "but they're in the same Pod!"

The most common multi-container bug is assuming that two containers in one Pod automatically see each other's files. They do not. Each container has its own volumeMounts list, and a shared volume only exists where both containers explicitly mount the same volume **name**.

manifests/90-sidecar-broken-volume.yaml has the app mounting varlog and the shipper mounting applogs - a name that is never declared:

```
kubectl apply -f manifests/90-sidecar-broken-volume.yaml
```

```
The Pod "depot-sidecar-broken" is invalid: spec.containers[1].volumeMounts[0].name: Not found:
"applogs"
```

```
kubectl -n kcna-lab04 get pod depot-sidecar-broken
```

```
Error from server (NotFound): pods "depot-sidecar-broken" not found
```

Diagnosis

- The error came from the API server at admission time, before any node was

involved. Contrast with Lab 01, where a missing **ConfigMap** was accepted and only failed later at the kubelet. The difference: a volumeMounts.name is an intra-object reference, so the API server can validate it without looking anything up. A ConfigMap name is an inter-object reference, and Kubernetes does not do referential integrity across objects.

- Read the field path: spec.containers[**1**].volumeMounts[**0**].name. Index 1

is the **second** container (shipper); index 0 is its first mount. The message tells you the offending container without naming it.

- Nothing was created. There is no Pod to delete, no events to read, no

half-broken state. Admission-time errors are the cheapest class of failure - which is the argument for running `kubectl apply --dry-run=server` in CI.

Now consider the **silent** version of this bug, which is far worse. If the author had instead mounted the right volume at the wrong path - say varlog at `/var/log/app` in the shipper while the app writes to `/var/log/depot` - the Pod would be perfectly valid, would start, and the shipper would simply tail a file that never appears. You would see:

NAME	READY	STATUS	RESTARTS	AGE
depot-sidecar-quiet	2/2	Running	0	3m

Two containers, both "healthy", and zero logs shipped. The diagnostic habit:

```
# 1. Does the writer actually see its file?
kubectl -n kcna-lab04 exec depot-sidecar -c app -- ls -l /var/log/depot
# 2. Does the reader see the SAME file at the path it is reading?
kubectl -n kcna-lab04 exec depot-sidecar -c shipper -- ls -l /var/log/depot
# 3. Compare the mounts the API server actually stored.
kubectl -n kcna-lab04 get pod depot-sidecar \
-o jsonpath='{range .spec.containers[*]}{.name}{": "}{.volumeMounts[*].mountPath}{"\n"}{end}'
```

```
total 4
-rw-r--r--    1 1000    3000          3122 Sep  5 16:20 access.log
total 4
-rw-r--r--    1 1000    3000          3122 Sep  5 16:20 access.log
app: /var/log/depot /seed
shipper: /var/log/depot
```

Same size, same timestamp, same path - that is what "sharing a volume" looks like when it is genuinely working.

Fix the broken manifest by making the names agree (either rename the volume to applogs in spec.volumes, or the mount to varlog), then re-apply. The working equivalent is already deployed as depot-sidecar.

7. Troubleshooting

Symptom	Likely cause	Fix
error: a container name must be specified for pod X, choose one of: [...]	kubectl logs/exec on a multi-container Pod without -c	Add -c <container>; use kubectl logs X --all-containers=true --prefix to see all at once
spec.containers[N].volumeMounts[M].name: Not found: "..." at apply time	A container mounts a volume name that spec.volumes does not declare	Make the names match; catch it earlier with kubectl apply --dry-run=server
Both containers Running, sidecar log is empty	Same volume mounted at different paths, or the writer buffers and never flushes	exec into each container and ls -l the path; compare .spec.containers[*].volumeMounts
Sidecar container CrashLoopBackOff with tail: can't open ...: No such file or directory	The sidecar started before the app created the file	Wait for the file in a loop (as 20-sidecar-logship.yaml does), or make it a native sidecar with an init container that creates the file
Error: failed to create containerd task: ... address already in use or nginx exits with bind() to 0.0.0.0:80 failed	Two containers in the Pod bound the same port - the network namespace is shared	Give each container a distinct port
Ambassador returns 502 Bad Gateway	The upstream Service has no ready endpoints, or the DNS name is wrong	kubectl get endpointslices -l kubernetes.io/service-name=<svc>; check the FQDN in the proxy config
Pod evicted with The node was low on resource: ephemeral-storage	An unbounded emptyDir filled the node disk	Set emptyDir.sizeLimit, and rotate or cap the log inside the container
Native sidecar behaves like a classic init container (Pod never starts the app)	restartPolicy: Always missing, or the cluster is older than v1.29	kubectl get pod X -o jsonpath='{.spec.initContainers[*].restartPolicy}'; check kubectl version
A Job with a sidecar never completes	The sidecar is a regular container, so the Pod stays Running forever	Move it to initContainers with restartPolicy: Always (Lab 08)

8. Cleanup

```
kubectl delete namespace kcna-lab04
```

```
namespace "kcna-lab04" deleted
```

9. What you learned

- A Pod is a shared execution context: one network namespace, one set of named volumes, one lifecycle - not "a container".
- Containers in a Pod do not share a filesystem root. Sharing happens only where two containers mount the same volume name, and mount options (readOnly, mountPath) are set per container.
- Sidecar adds a capability; adapter reshapes an output to a standard; ambassador owns outbound connectivity on localhost. The mechanics are the same; the intent is what you document.
- `kubectl logs/exec` need `-c` on multi-container Pods; the error helpfully lists the container names.
- One shared loopback means containers reach each other on 127.0.0.1 - and means they cannot both bind the same port.
- Native sidecars (initContainers + restartPolicy: Always, v1.29+, GA v1.33) give the start-order and shutdown-order guarantees that ordinary sidecars lack, and they let Jobs complete.
- Intra-object references are validated at admission; inter-object references are not validated at all. `--dry-run=server` catches the first class in CI.

10. Further reading

- [Pods - Workload resources](#)
- [Sidecar Containers](#)
- [Init Containers](#)
- [Kubernetes 1.28: Introducing native sidecar containers](#)
- [Communicate Between Containers in the Same Pod Using a Shared Volume](#)
- [Volumes - emptyDir](#)
- [Logging Architecture - sidecar container patterns](#)
- [EndpointSlices](#)